



第四章 快速傅里叶变换(FFT)

4.1 DFT运算的特点

4.2 按时间抽取

4.3 按频率抽取

4.4 IDFT的运算方法

4.5 任意基数的算法

4.6 Chirp-Z变换

4.7 线性卷积的FFT算法

4.8 FFT的流水线工作原理



快速傅里叶变换(FFT)是计算DFT的一种快速高效的算法。

有限长序列 $x(n)$ 的DFT是 $X(k)$ 对序列频谱 $X(e^{j\omega})$ 的采样。所以有限长序列的DFT本身可以完全表达序列的频谱，可以直接对信号进行频谱分析。因此DFT在数字信号处理技术中占有很重要的地位。



虽然频谱分析和DFT运算很重要，但在很长一段时间里，由于DFT运算量太大，因此它并没有得到真正的运用，频谱分析也大多采用模拟信号滤波的方法解决。直到1965年美国IBM公司的库利和图基这两位科学家首次提出DFT运算的一种快速算法以后，情况才发生了根本变化，人们开始认识到DFT运算的一些内在规律，从而很快地发展和完善了一套高速有效的运算方法——快速付里叶变换(FFT)算法。FFT的出现，使DFT的运算时间缩短1-2个数量级，FFT的问世使得DFT在实际中得到广泛应用。



§4.1 DFT运算的特点

有限长序列 $x(n)$ 进行一次DFT运算所需的运算量:

$$X(k) = \text{DFT}[x(n)] = \sum_{n=0}^{N-1} x(n) w_N^{nk} \quad k = 0, 1, \dots, N-1$$

一般 $x(n)$ 和 w_N^{nk} 都是复数, $X(k)$ 也是复数, 所以计算一次DFT的运算量是:

N 次复数相乘和 $(N-1)$ 次复数相加

$X(k)$ 一共有 N 个点, 因此完成全部DFT的运算量是:

N^2 次复数相乘和 $N(N-1)$ 次复数相加



在这些运算中，乘法比加法运算复杂，尤其是复数相乘，每个复数相乘包括4个实数相乘和2个实数相加，例：

$$X(k) = \sum_{n=0}^{N-1} \left\{ \left(R_e[x(n)]R_e[w_N^{nk}] - I_m[x(n)]I_m[w_N^{nk}] \right) + j \left(R_e[x(n)]I_m[w_N^{nk}] + I_m[x(n)]R_e[w_N^{nk}] \right) \right\}$$

又因每个复数相加包括2个实数相加,所以,每计算一个 $X(k)$ 要进行 $4N$ 次实数相乘和 $2N+2(N-1)=2(2N-1)$ 次实数相加，因此，整个DFT运算需要 $4N^2$ 实数相乘和 $2N(2N-1)$ 次实数相加。

一般情况下，将系数 w_N^{nk} 都看作为复数，而不考虑简单运算的特例。



从上面的分析看到，在DFT计算中，不论是乘法和加法，运算量均与 N^2 成正比。因此， N 较大时，运算量十分可观。例如，计算 $N=10$ 点的DFT，需要100次复数相乘，而 $N=1024$ 点时，需要1048576次复数乘法，如果要求实时处理，则要求有很高的计算速度才能完成上述计算量。

反变换IDFT与DFT的运算结构相同，只是多乘一个常数 $1/N$ ，所以二者的计算量相同。



考察DFT的运算特点发现,利用以下两个特性可减少运算量:

系数 $w_N^{nk} = e^{-j\frac{2\pi}{N}nk}$ 是一个周期函数, 利用它的周期性和对称性可改进运算量, 提高计算效率。

➤ 周期性 $w_N^{n(N-k)} = w_N^{k(N-n)} = w_N^{-nk}$

➤ 对称性 $w_N^{(nk+N/2)} = -w_N^{nk} \quad w_N^{N/2} = -1$

我们利用系数 w_N^{nk} 的周期性和对称性, 合并DFT运算中某些项, 从而减少运算量。



我们可以利用系数 w_N^{nk} 的周期性和对称性，使DFT运算尽量分解为更少点数的DFT运算。由于其运算工作量与 N^2 成正比，故而 N 越小越好。因此可以把长度为 N 点的大点数DFT运算依次分解为若干个小点数的DFT来运算。

FFT正是基于这样的基本思想来提高DFT的运算速度。它的算法形式有很多种，基本上可以分为两大类，即时间抽取法和频率抽取法。



§4.2 按时间抽取

为了保证 N 点DFT的运算分解，首先假设 N 是2的整幂次方，即

$$N = 2^M \quad \text{其中，} M \text{ 为正整数} \quad (4-1)$$

这样就可以将序列 $x(n)$ 按序号 n 的奇、偶分解为两个 $\frac{N}{2}$ 点的子序列：

$$\begin{cases} x(2r) = x_1(r) & \text{偶数项组} \\ x(2r+1) = x_2(r) & \text{奇数项组} \end{cases} \quad r = 0, 1, \dots, \frac{N}{2} - 1 \quad (4-2)$$



DFT运算也相应分为两组：

$$X(k) = \text{DFT}[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{nk}, \quad 0 \leq k \leq N-1$$

$$= \sum_{\text{偶数组 } n=0}^{N-2} x(n) w_N^{nk} + \sum_{\text{奇数组 } n=1}^{N-1} x(n) w_N^{nk}$$

$$= \sum_{r=0}^{N/2-1} x(\textcolor{red}{2}r) w_N^{2rk} + \sum_{r=0}^{N/2-1} x(\textcolor{blue}{2}r + 1) w_N^{(\textcolor{blue}{2}r + 1)k}$$

$$= \sum_{r=0}^{N/2-1} x_1(r) w_N^{2rk} + w_N^k \sum_{r=0}^{N/2-1} x_2(\textcolor{blue}{r}) w_N^{2rk}$$



我们注意到 W_N^{2n} 可以转变为 $W_{N/2}^n$ ，即

$$w_{\textcolor{red}{N}}^{2rk} = e^{-j\frac{2\pi}{N}2rk} = e^{-j\frac{2\pi}{\textcolor{red}{N}/2}rk} = w_{\textcolor{red}{N}/2}^{rk}$$

$$\begin{aligned} \text{那么, } X(k) &= \sum_{r=0}^{N/2-1} x_1(r) w_{\textcolor{red}{N}/2}^{rk} + w_N^k \sum_{r=0}^{N/2-1} x_2(r) w_{\textcolor{red}{N}/2}^{rk} \\ &= \sum_{r=0}^{N/2-1} x(2r) W_{\textcolor{red}{N}/2}^{rk} + w_N^k \sum_{r=0}^{N/2-1} x(2r+1) W_{\textcolor{red}{N}/2}^{rk} \\ &= X_1(k) + w_N^k X_2(k) \end{aligned} \quad (4-3)$$

其中, $X_1(k)$ 和 $X_2(k)$ 分别是 $x_1(r)$ 及 $x_2(r)$ 的 $\frac{N}{2}$ 点的DFT, 即

$$X_1(k) = \sum_{r=0}^{N/2-1} x_1(r) w_{\textcolor{red}{N}/2}^{rk} = \sum_{r=0}^{N/2-1} x(2r) w_{\textcolor{red}{N}/2}^{rk} \quad (4-4)$$

$$X_2(k) = \sum_{r=0}^{N/2-1} x_2(r) w_{\textcolor{red}{N}/2}^{rk} = \sum_{r=0}^{N/2-1} x(2r+1) w_{\textcolor{red}{N}/2}^{rk} \quad (4-5)$$



$X_1(k)$ 、 $X_2(k)$ 都只有 $N/2$ 个点, 而 $X(k)$ 有 N 个点, 要用 $X_1(k)$ 、 $X_2(k)$ 表达全部 $X(k)$ 值还需应用 W 系数的周期特性,

$$\text{即} \quad w_{N/2}^{r(N/2+k)} = w_{N/2}^{rk}$$

$$\text{那么,} \quad X_1(N/2+k) = \sum_{r=0}^{N/2-1} x_1(r) w_{N/2}^{r(N/2+k)} = \sum_{r=0}^{N/2-1} x_1(r) w_{N/2}^{rk}$$

$$\text{即} \quad X_1(N/2+k) = X_1(k) \quad (4-7)$$

$$X_2(N/2+k) = X_2(k) \quad (4-8)$$

同时考虑到 W 的对称特性, 即 $w_N^{(N/2+k)} = w_N^{N/2} \cdot w_N^k = -w_N^k$

综上, 我们可以将 $X(k)$ 表达为前后两部分。



可见，一个 N 点的DFT被分解为前后两个 $N/2$ 点的DFT，这两个 $N/2$ 点的DFT再合成为一个 N 点DFT。

$$\left\{ \begin{array}{l} X(k) = X_1(k) + W_N^k X_2(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1 \quad (4-9) \\ X(k + \frac{N}{2}) = X_1(k + \frac{N}{2}) + W_N^{(\frac{N}{2} + k)} X_2(k + \frac{N}{2}) \\ \quad = X_1(k) - W_N^k X_2(k) \\ \quad \quad \quad k = 0, 1, \dots, \frac{N}{2} - 1 \quad (4-10) \end{array} \right.$$

以上两个表达式说明：只要我们计算出 $N/2$ 个点的所有 $X_1(k)$ 和 $X_2(k)$ 的值，就等于求出 N 点的 $X(k)$ 值，这使得DFT的运算量减少了约一半。



上述两个公式的运算过程可用专用蝶形图表示：

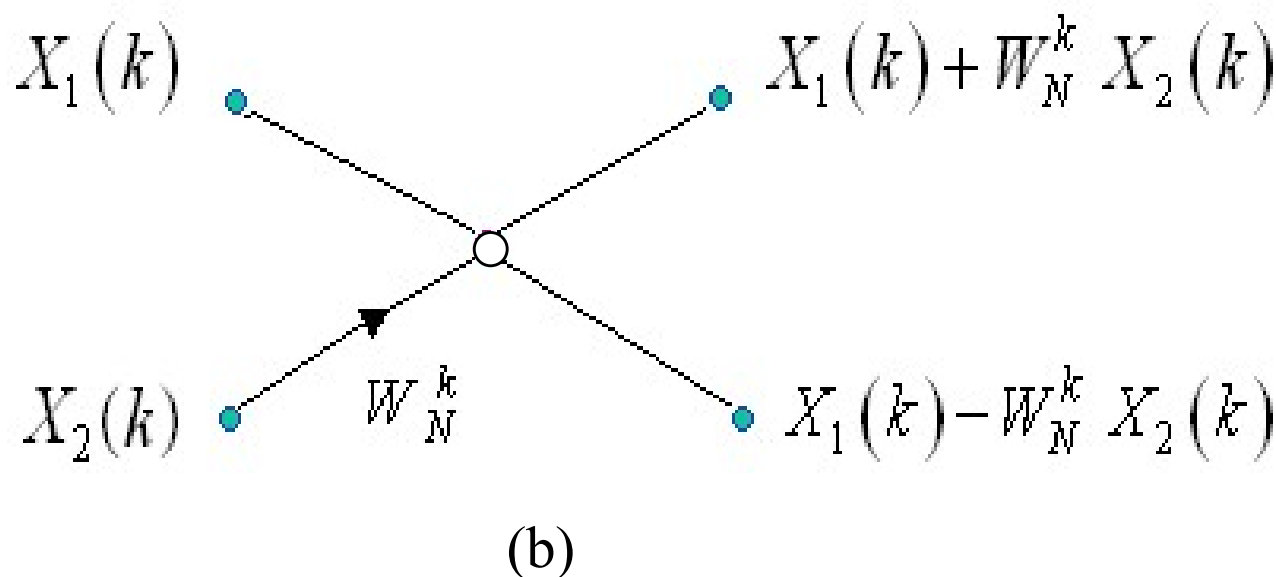
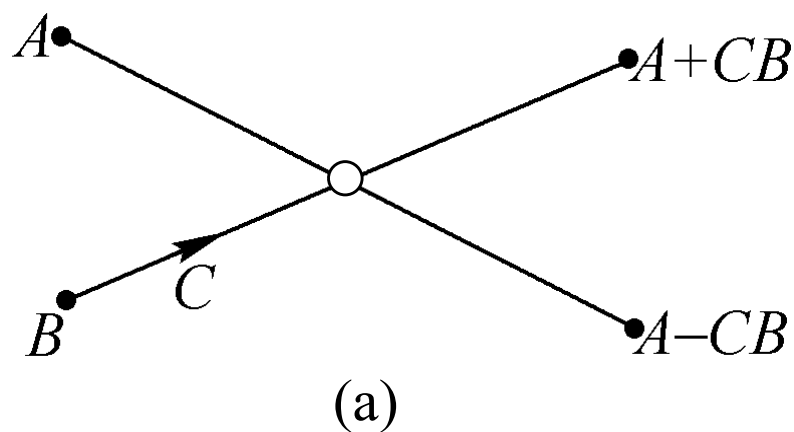


图4.2 蝶形运算流图符号



以 $N = 2^3 = 8$ 为例，可得8点DFT一次时域抽取分解运算流程图：

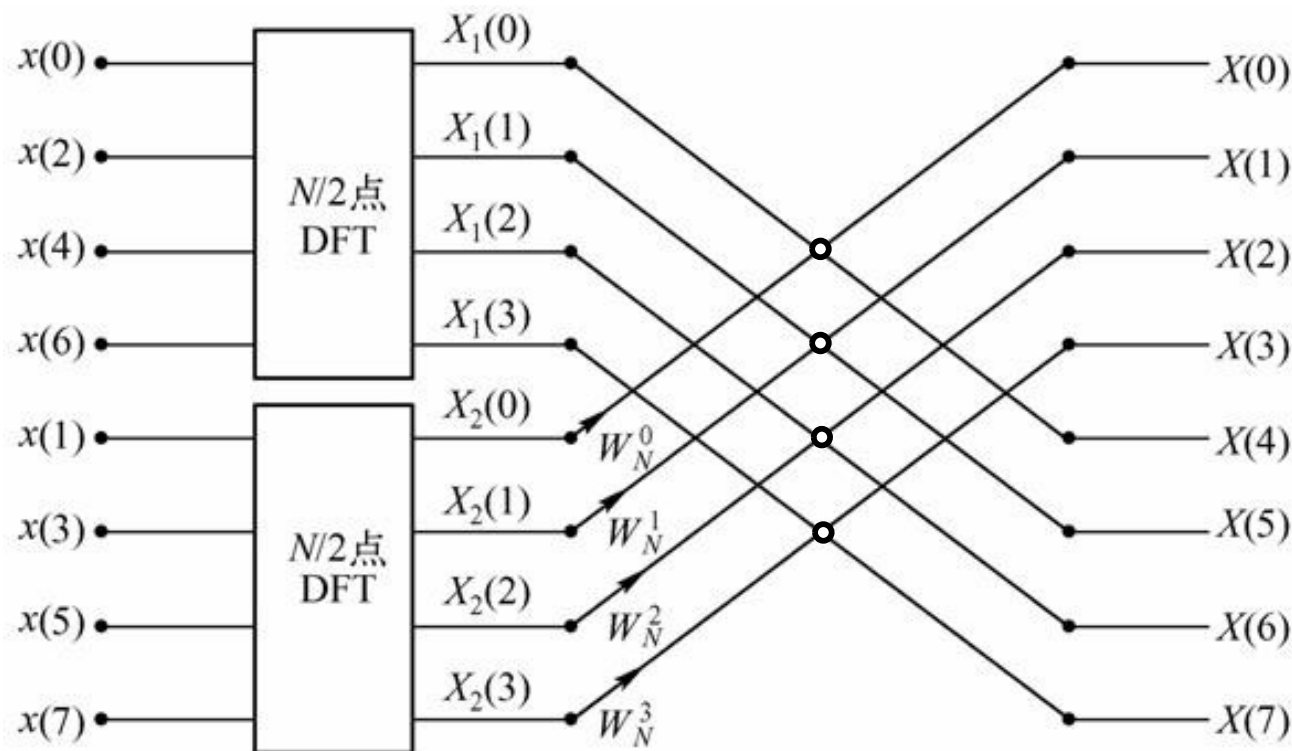


图4.3 按时间抽取将一个 N 点DFT分解为两个 $N/2$ 点DFT ($N=8$)



由于 $N = 2^M$, $N/2$ 仍然是偶数, 因此 $X_1(k)$ 和 $X_2(k)$ 可以继续分解下去, 可将 $N/2$ 点的子序列再按奇、偶项分解, 一直到最后分解成两两点的DFT为止。

$$\left. \begin{array}{l} \text{四个} \\ \frac{N}{4} \text{点} \\ \text{DFT} \end{array} \right\} \begin{array}{ll} \text{偶序列中的偶序列} & x_1(2l) = x_3(l) \\ \text{偶序列中的奇序列} & x_1(2l+1) = x_4(l) \\ \text{奇序列中的偶序列} & x_2(2l) = x_5(l) \\ \text{奇序列中的奇序列} & x_2(2l+1) = x_6(l) \end{array}$$

$$l = 0, 1, \dots, \frac{N}{4} - 1$$



$$\begin{cases} x_1(2l) = x_3(l) \\ x_1(2l+1) = x_4(l) \end{cases} \quad l = 0, 1, \dots, \frac{N}{4} - 1$$

$$\begin{aligned} X_1(k) &= \sum_{l=0}^{\frac{N}{4}-1} x_1(2l) W_{N/2}^{2lk} + \sum_{l=0}^{\frac{N}{4}-1} x_1(2l+1) W_{N/2}^{(2l+1)k} \\ &= \sum_{l=0}^{\frac{N}{4}-1} x_3(l) W_{N/4}^{lk} + W_{N/2}^k \sum_{l=0}^{\frac{N}{4}-1} x_4(l) W_{N/4}^{lk} \\ &= X_3(k) + W_{N/2}^k X_4(k) \quad k = 0, 1, \dots, \frac{N}{4} - 1 \end{aligned}$$



且

$$X_1\left(\frac{N}{4} + k\right) = X_3(k) - W_{N/2}^k X_4(k) \quad k = 0, 1, \dots, \frac{N}{4} - 1$$

式中：

$$X_3(k) = \sum_{l=0}^{\frac{N}{4}-1} x_3(l) W_{N/4}^{lk}$$

$$X_4(k) = \sum_{l=0}^{\frac{N}{4}-1} x_4(l) W_{N/4}^{lk}$$

图6-4给出 $N=8$ 时，将一个 $N/2$ 点DFT分解成两个 $N/4$ 点DFT，由这两个 $N/4$ 点DFT组合成一个 $N/2$ 点DFT的流图。

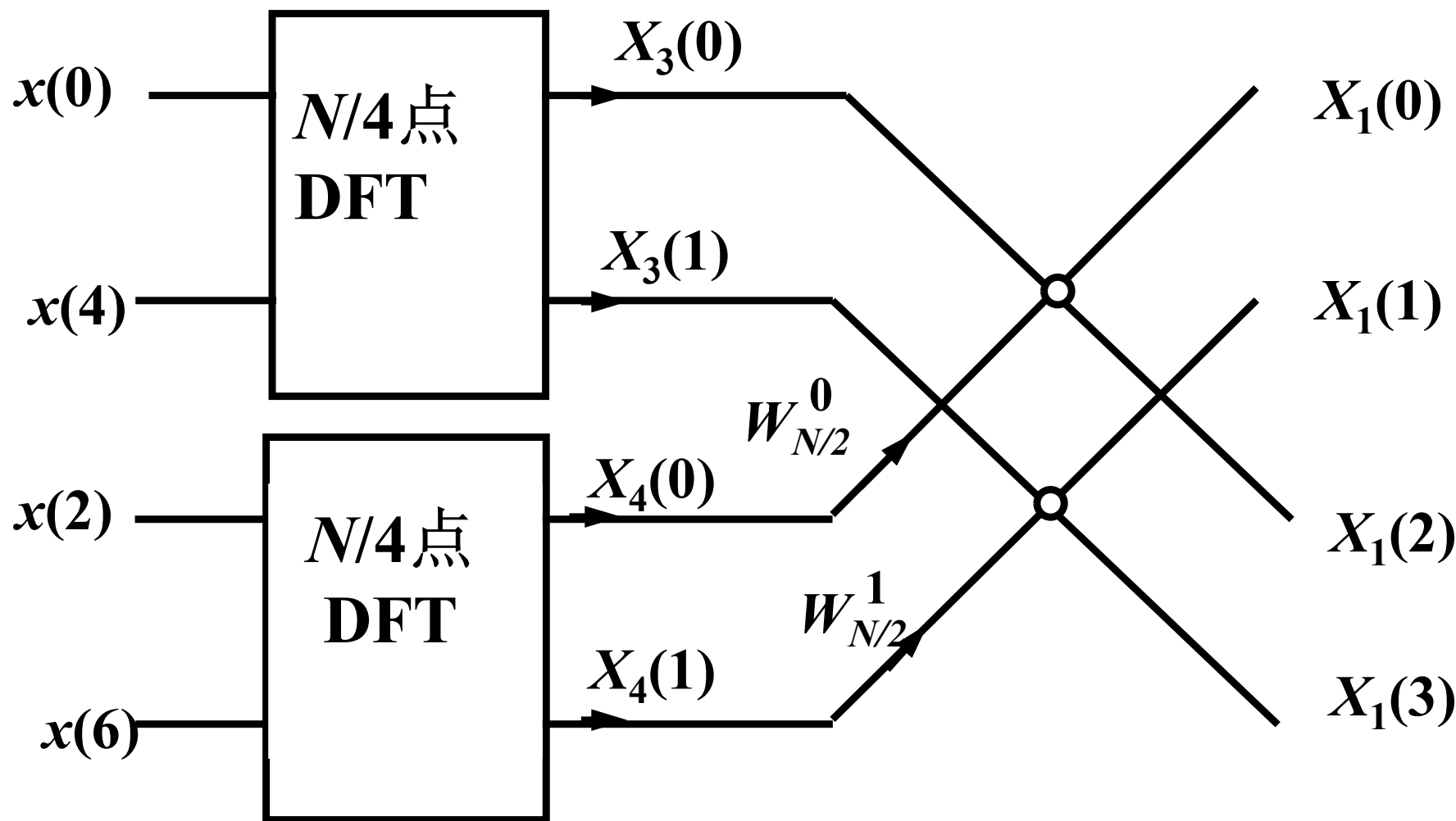


图 4-4 $N/2$ 点DFT分解为两个 $N/4$ 点DFT

同理 $X_2(k)$ 再分解为两个 $N/4$ 点的DFT $X_5(k)$ 、 $X_6(k)$ 。¹⁹



$X_2(k)$ 也可进行同样的分解:

$$\begin{cases} X_2(k) = X_5(k) + W_{N/2}^k X_6(k) \\ X_2\left(\frac{N}{4} + k\right) = X_5(k) - W_{N/2}^k X_6(k) \end{cases} \quad k = 0, 1, \dots, \frac{N}{4} - 1$$

式中:

$$X_5(k) = \sum_{l=0}^{\frac{N}{4}-1} x_5(l) W_{N/4}^{lk} = \sum_{l=0}^{\frac{N}{4}-1} x_2(2l) W_{N/4}^{lk}$$

$$X_6(k) = \sum_{l=0}^{\frac{N}{4}-1} x_6(l) W_{N/4}^{lk} = \sum_{l=0}^{\frac{N}{4}-1} x_2(2l+1) W_{N/4}^{lk}$$



以 $N=8$ 为例，由四个2点DFT组成的8点DFT如下图所示：

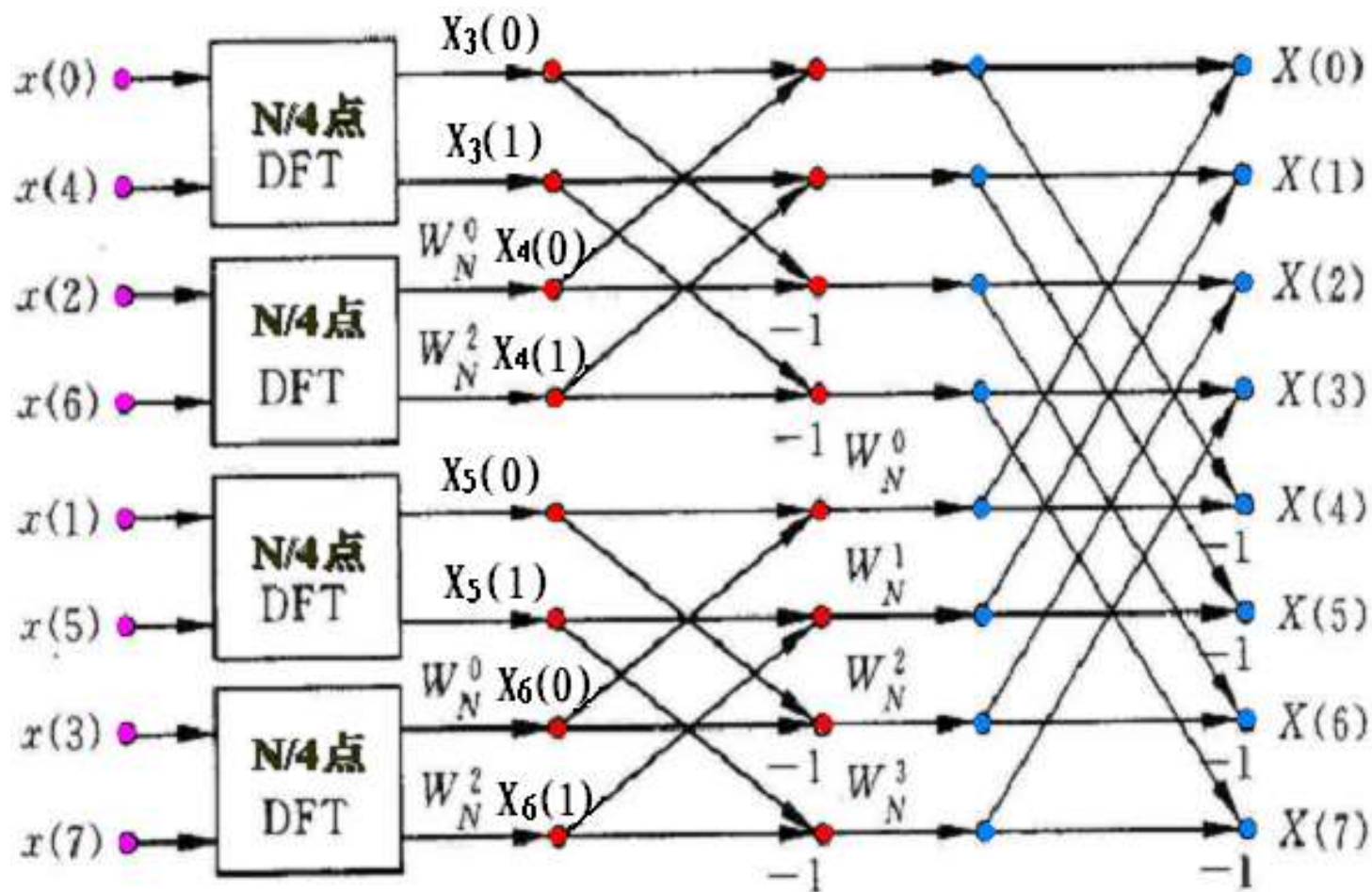


图4.5



在四个2点的DFT基础上，将剩下的2点DFT用一个蝶形结表示，可得一个完整的按时间抽取运算的8点DFT流图：

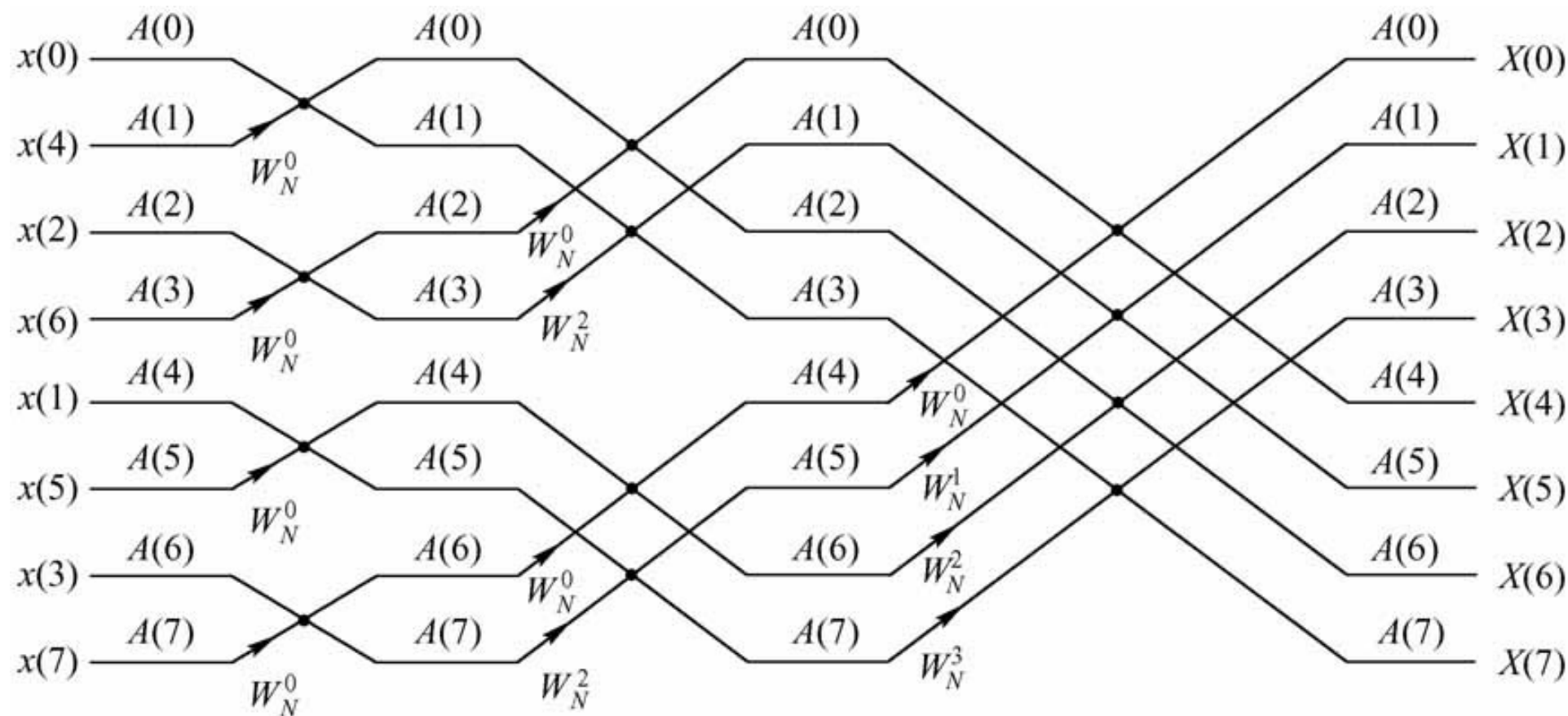


图4.6



这种方法，由于每一步分解都是按输入序列是属于偶数还是奇数来抽取的，所以称为“按时间抽取法”或“时间抽取法”。

对于任何一个2的整数幂 $N = 2^M$ ，总是可以通过 M 次的分解最后完全成为2点的DFT运算。这样的 M 次分解，就构成了从 $x(n)$ 到 $X(k)$ 的 M 级运算过程。每一级运算都是由 $N/2$ 个蝶形运算构成，需要 $N/2$ 次复乘和 N 次复加，因而 M 级运算的运算量为：

$$\text{复乘数} \quad m_F = \frac{N}{2} \bullet M = \frac{N}{2} \log_2 N$$

$$\text{复加数} \quad a_F = N \bullet M = N \log_2 N$$

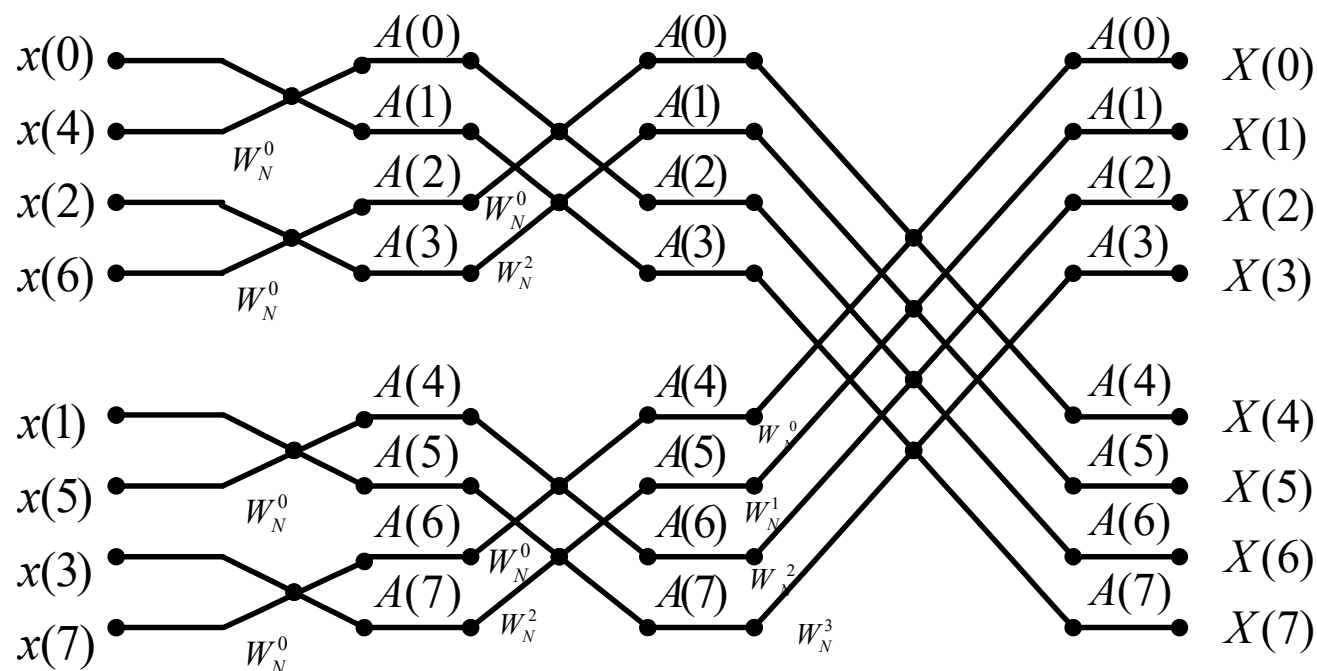
结论：用时间抽取法所需的运算工作量不论是复数乘还是复数加都与 $N \log_2 N$ 成正比。



1. 原位运算

所谓原位运算，就是当数据输入到存储器中以后，每一级运算的结果仍然储存在这同一组存储器中，直到最后输出，中间无需其它的存储器。

时间抽取法的每一级运算都有 $N/2$ 个蝶形运算，它是由一次加权运算和一次加减运算构成的；它可以采用原位运算的方式，将整个变换运算分为M级，每一级运算均可在原位进行，这种原位运算的结构可以节省存储单元，降低设备成本，但缺点是不能按自然顺序存放。





2. 码位倒置

观察上图的同址计算结构，发现当运算完成后，FFT的输出 $X(k)$ 按正常顺序排列在存储单元中，即按 $X(0)$, $X(1)$, ..., $X(7)$ 的顺序排列，但是这时输入 $x(n)$ 却不是按自然顺序存储的，而是按 $x(0)$, $x(4)$, ..., $x(7)$ 的顺序存入存储单元，看起来好像是“混乱无序”的，实际上是有规律的，我们称之为倒位序。

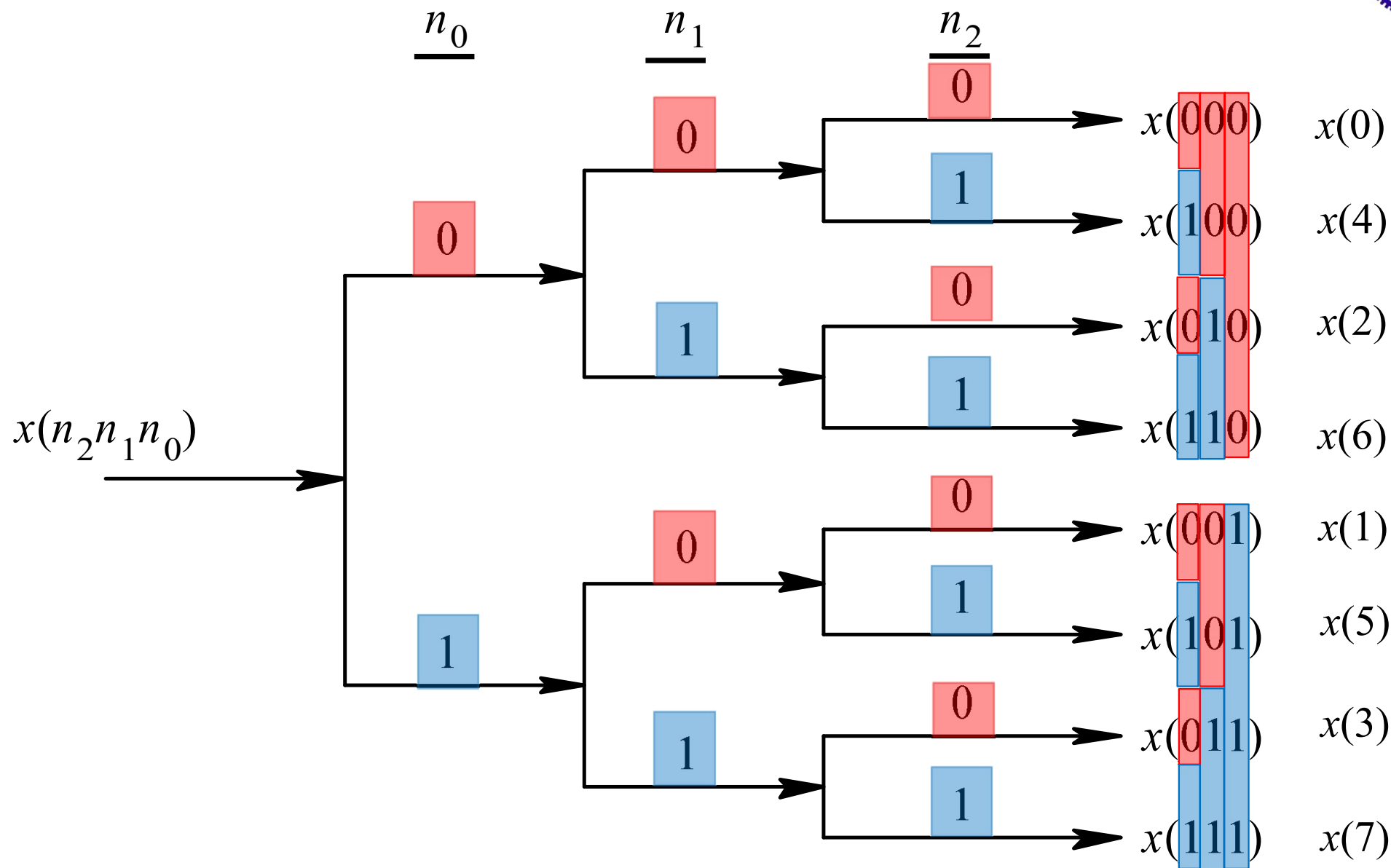


图4-7 倒位序的形成



造成倒位序的原因是输入 $x(n)$ 按标号 n 的偶奇的不断分组。如果 n 用二进制数表示为 $(n_2n_1n_0)_2$ （当 $N=8=2^3$ 时，二进制为三位），第一次分组，由图4.3看出， n 为偶数（相当于 n 的二进制数的最低位 $n_0=0$ ）在上半部分， n 为奇数（相当于 n 的二进制数的最低位 $n_0=1$ ）在下半部分。下一次则根据次最低位 n_1 为“0”或是“1”来分偶奇（而不管原来的子序列是偶序列还是奇序列），如此继续分下去，直到最后 N 个长度为1的子序列。图4-7的树状图描述了这种分成偶数子序列和奇数子序列的过程。



一般实际运算中，总是先按自然顺序将输入序列存入存储单元，为了得到倒位序的排列，我们通过变址运算来完成。如果输入序列的自然顺序号 I 用二进制数（例如 $n_2n_1n_0$ ）表示，则其倒位序 J 对应的二进制数就是 $(n_0n_1n_2)$ ，这样，在原来自然顺序时应该放 $x(I)$ 的单元，现在倒位序后应放 $x(J)$ 。例如， $N=8$ 时， $x(3)$ 的标号是 $I=3$ ，它的二进制数是011，倒位序的二进制数是110，即 $J=6$ ，所以原来存放在 $x(011)$ 单元的数据现在应该存放在 $x(110)$ 内。表4-1列出了 $N=8$ 时的自然顺序二进制数以及相应的倒位序二进制数。



表4-1 $N=8$ 时的自然顺序二进制数和相应的倒位序二进制数

自然顺序 (I)	二进制数	倒位序二进制数	倒位序 (J)
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7



由表4-1可见，自然顺序数 I 增加1，是在顺序数的二进制数最低位加1，向左进位。而倒序数 J 则是在二进制数最高位加1，逢2向右进位。例如，在(000)最高位加1，则得(100)，再在(100)最高位加1，向右进位，则得(010)。因(100)最高位为1，所以最高位加1要向次高位进位，其实质是将最高位变为0，再在次高位加1。用这种算法，可以从当前任一倒序值求得下一个倒序值。